

Probabilidades gaussianas y entropía

¿Por qué las distancias no son suficientes?

En la Parte 1 aprendimos a calcular la distancia entre todas las observaciones del conjunto de datos.

Obtuvimos el siguiente vector de distancias:

$$[1.4142 \quad 5.0000 \quad 6.4031 \quad 3.6056 \quad 5.0000 \quad 1.4142]$$

También construimos la matriz de distancias

$$D = \begin{bmatrix} 0 & 1.4142 & 5.0000 & 6.4031 \\ 1.4142 & 0 & 3.6056 & 5.0000 \\ 5.0000 & 3.6056 & 0 & 1.4142 \\ 6.4031 & 5.0000 & 1.4142 & 0 \end{bmatrix}$$

¿Podemos utilizar directamente estas distancias?

La respuesta es **no**.

¿Por qué?

Porque las distancias dependen completamente de la escala de los datos.

Por ejemplo,

Distancia	Interpretación
0.5	Muy cercana
5	¿Cercana o lejana?
500	¿Lejana? Depende del problema

Una misma distancia puede representar conceptos completamente diferentes dependiendo del conjunto de datos.

Necesitamos una medida independiente de la escala

En lugar de trabajar con distancias,

t-SNE trabaja con **probabilidades**.

Las probabilidades siempre pertenecen al intervalo

$$0 \leq P \leq 1$$

y pueden compararse entre diferentes conjuntos de datos.

De las distancias a las probabilidades

Supongamos que una observación tiene tres vecinos.

Vecino	Distancia
A	1
B	2
C	6

Claramente,

A es el vecino más cercano.

C es el más lejano.

La idea de t-SNE consiste en convertir esas distancias en probabilidades.

En lugar de decir

"A está a una distancia igual a 1"

diremos

"Existe una alta probabilidad de que A sea vecino."

Después de la transformación podríamos obtener

Vecino	Probabilidad
A	0.72
B	0.24
C	0.04

Observemos que

$$0.72 + 0.24 + 0.04 = 1$$

Ahora disponemos de una distribución de probabilidad.

Interpretación

Las probabilidades describen la importancia relativa de cada vecino.

Un vecino cercano recibe una probabilidad grande.

Un vecino lejano recibe una probabilidad pequeña.

En consecuencia,

las probabilidades contienen mucha más información que las distancias originales.

La distribución Gaussiana

¿Cómo convertir una distancia en una probabilidad?

t-SNE utiliza una función Gaussiana.

$$f(x) = e^{-x}$$

Sin embargo,

la variable que nos interesa es la distancia al cuadrado.

Por ello utilizamos

$$e^{-\frac{d_{ij}^2}{2\sigma_i^2}}$$

donde

- d_{ij} es la distancia entre los puntos i y j .
- σ_i controla el ancho de la campana de Gauss.

Ejemplo

Supongamos

$$\sigma = 1$$

Entonces

Distancia	Valor Gaussiano
0	1.0000
1	0.6065
2	0.1353

Distancia	Valor Gaussiano
3	0.0111
4	0.0003

Observemos que

la probabilidad disminuye muy rápidamente a medida que aumenta la distancia.

Idea principal

Los vecinos cercanos reciben valores grandes.

Los vecinos lejanos reciben valores muy pequeños.

Esto es exactamente lo que deseamos para representar relaciones de vecindad.

Diapositiva 24 - Probabilidades condicionales

Sea una observación

$$\mathbf{x}_i$$

Deseamos conocer la probabilidad de escoger como vecino al punto

$$\mathbf{x}_j$$

La probabilidad condicional utilizada por t-SNE es

$$P_{j|i} = \frac{e^{-\frac{d_{ij}^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{-\frac{d_{ik}^2}{2\sigma_i^2}}}$$

Observemos el denominador.

$$\sum_{k \neq i} e^{-\frac{d_{ik}^2}{2\sigma_i^2}}$$

Su función consiste en normalizar todas las probabilidades.

Como consecuencia,

$$\sum_{j \neq i} P_{j|i} = 1$$

Interpretación

Si

$$P_{j|i} = 0.80$$

significa que

el punto j tiene una probabilidad del 80% de ser considerado vecino del punto i .

Si

$$P_{j|i} = 0.01$$

el punto está muy alejado de i .

En las siguientes diapositivas aprenderemos cómo calcular estas probabilidades utilizando únicamente operaciones tensoriales en MXNet y veremos por qué cada fila necesita un valor diferente de

$$\sigma_i.$$

¿Qué representa σ ?

En la diapositiva anterior definimos la probabilidad condicional como

$$P_{j|i} = \frac{e^{-\frac{d_{ij}^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{-\frac{d_{ik}^2}{2\sigma_i^2}}}$$

El parámetro

$$\sigma_i$$

es la desviación estándar de una distribución Gaussiana centrada en la observación

$$\mathbf{x}_i.$$

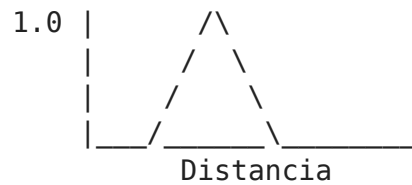
Interpretación geométrica

Si σ es pequeño,

la campana Gaussiana es muy angosta.

Únicamente los vecinos muy cercanos reciben probabilidades elevadas.

Probabilidad

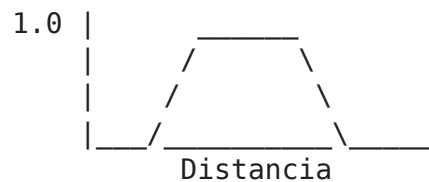


Si σ es grande,

la campana es más ancha.

Muchos vecinos reciben probabilidades similares.

Probabilidad



Cada observación tendrá su propio valor

$$\sigma_i$$

para adaptarse a la densidad local de los datos.

Introducción del parámetro β

Aunque la formulación con

$$\sigma$$

es intuitiva,

no es la más conveniente para implementar el algoritmo.

Por ello se define un nuevo parámetro

$$\beta_i = \frac{1}{2\sigma_i^2}$$

Sustituyendo esta definición en la distribución Gaussiana,

$$e^{-\frac{d_{ij}^2}{2\sigma_i^2}} = e^{-\beta_i d_{ij}^2}$$

obtenemos una expresión mucho más sencilla.

La probabilidad condicional pasa a escribirse como

$$P_{j|i} = \frac{e^{-\beta_i d_{ij}^2}}{\sum_{k \neq i} e^{-\beta_i d_{ik}^2}}$$

Ventaja

Ahora únicamente debemos ajustar un único parámetro

$$\beta_i$$

en lugar de trabajar directamente con

$$\sigma_i.$$

Esta es exactamente la formulación utilizada por la implementación tensorial.

Correspondencia con la implementación MXNet

Observemos nuevamente una de las instrucciones principales.

```
my $row_P =
  nd->exp(-$sqdistances_f64 * $beta);
```

Recordemos la ecuación matemática

$$e^{-\beta_i d_{ij}^2}$$

Comparando ambos términos,

Matemáticas	MXNet
d_{ij}^2	<code>sqdistances_f64</code>
β_i	<code>beta</code>
e^x	<code>nd->exp()</code>

La correspondencia es inmediata.

Vectorización

Supongamos

$$\beta = \begin{bmatrix} 0.5 \\ 0.8 \\ 0.3 \\ 0.6 \end{bmatrix}$$

Cada fila posee su propio valor de

$$\beta_i.$$

Gracias al **broadcasting**,

MXNet multiplica automáticamente cada fila de

`sqdistances_f64`

por el correspondiente valor de

$$\beta_i$$

sin utilizar ningún bucle explícito.

Toda la matriz de probabilidades gaussianas se calcula en una sola operación tensorial.

Ejemplo completo de una fila de probabilidades

Utilicemos las distancias al cuadrado obtenidas anteriormente para el punto A.

$$[0 \quad 2 \quad 25 \quad 41]$$

La distancia a sí mismo no participa en el cálculo.

Consideraremos únicamente

$$[2 \quad 25 \quad 41]$$

Supongamos

$$\beta = 0.25$$

Calculamos los exponentes.

Distancia ²	Exponente
2	−0.25
25	−0.5
25	−6.25
41	−10.25

Aplicamos la función exponencial.

Distancia ²	$e^{-\beta d^2}$
2	0.6065
25	0.00193
41	0.000035

La suma es

$$0.6065 + 0.00193 + 0.000035 = 0.608465$$

Normalizando,

$$P_{j|A} = \frac{e^{-\beta d^2}}{\sum e^{-\beta d^2}}$$

obtenemos

Vecino	Probabilidad
B	0.9968
C	0.0032
D	0.00006

Interpretación

Con este valor de

$$\beta$$

el algoritmo considera que

B es prácticamente el único vecino de A.

En la siguiente sección veremos cómo escoger automáticamente el valor adecuado de

$$\beta$$

mediante la **entropía de Shannon** y la **perplejidad**.

¿Qué ocurre si β es demasiado pequeño?

Recordemos la distribución de probabilidad.

$$P_{j|i} = \frac{e^{-\beta d_{ij}^2}}{\sum_{k \neq i} e^{-\beta d_{ik}^2}}$$

Analicemos nuevamente las distancias desde el punto **A**.

Vecino	Distancia ²
B	2
C	25
D	41

Supongamos que

$$\beta = 0.01$$

Calculamos

Distancia ²	Exponente	Valor
2	-0.02	0.9802
25	-0.25	0.7788
41	-0.41	0.6637

La suma es

$$2.4227$$

Normalizando,

Vecino	Probabilidad
B	0.4047
C	0.3215
D	0.2738

Observación

Las tres probabilidades son casi uniformes (vecindad demasiado amplia).

El algoritmo prácticamente considera que **todos los vecinos son igualmente importantes**.

Se pierde la información de vecindad.

¿Qué ocurre si β es demasiado grande?

Ahora supongamos

$$\beta = 1$$

Volvemos a calcular

$$e^{-\beta d^2}$$

Distancia ²	Valor
2	0.135335
25	0.0000000000139
41	0.00000000000015

Normalizando,

Vecino	Probabilidad
B	0.9999999999
C	0.0000000001
D	≈ 0

Observación

Toda la probabilidad queda concentrada en un único vecino (vecindad demasiado estrecha).

El resto prácticamente desaparece.

Esto tampoco representa correctamente la estructura del conjunto de datos.

Necesitamos un valor de

$$\beta$$

que produzca un equilibrio entre ambos extremos.

Necesitamos medir la dispersión

Comparemos las dos distribuciones obtenidas.

Caso 1

$$\beta = 0.01$$

$$[0.405, 0.322, 0.274]$$

Las probabilidades están bastante repartidas.

Caso 2

$$\beta = 1$$

$$[0.999999, 0.000001, 0]$$

Toda la probabilidad está concentrada.

Pregunta

¿Cómo podemos medir matemáticamente

qué tan repartida está una distribución de probabilidad?

Necesitamos una función que sea

- grande cuando las probabilidades sean parecidas;
- pequeña cuando una sola probabilidad domine.

Esa función ya existe.

Se denomina **Entropía de Shannon**.

Entropía de Shannon

Sea una distribución

$$P = \{P_1, P_2, \dots, P_n\}$$

La entropía se define como

$$H(P) = - \sum_i P_i \log(P_i)$$

Interpretación

La entropía mide la incertidumbre de una distribución.

- Entropía grande

Las probabilidades están repartidas.

0.33 0.33 0.34

No sabemos cuál vecino será escogido.

- Entropía pequeña

0.99 0.01 0.00

La elección del vecino es casi segura.

Ejemplo

Sea

$$P = [0.5, 0.3, 0.2]$$

Entonces

$$H(P) = -(0.5 \log 0.5 + 0.3 \log 0.3 + 0.2 \log 0.2)$$

Calculando,

$$H(P) = 1.0297$$

En las siguientes diapositivas veremos cómo t-SNE utiliza esta entropía para definir la **Perplejidad**, que será el criterio utilizado para ajustar automáticamente el valor de

$$\beta$$

de cada observación.

Ver curiosidad algebraica: ¿De dónde proviene la fórmula de la entropía utilizada en MXNet?

La Perplejidad

Hasta ahora hemos aprendido que cada observación posee una distribución de probabilidades

$$P_{j|i}.$$

También vimos que la dispersión de dicha distribución puede medirse mediante la entropía

$$H(P).$$

Sin embargo,

la entropía no resulta fácil de interpretar.

Por ello, t-SNE utiliza una cantidad equivalente denominada **Perplejidad**.

Definición

La perplejidad se define como

$$\text{Perplexity}(P) = e^{H(P)}$$

donde

$$H(P)$$

es la entropía de Shannon.

Interpretación

La perplejidad puede entenderse como

el número efectivo de vecinos considerados por una observación.

Mientras mayor sea la perplejidad,

más vecinos influirán en la posición final del punto.

Interpretación intuitiva de la Perplejidad

Supongamos tres distribuciones distintas.

Caso 1

$$P = [1, 0, 0]$$

La entropía es

$$H = 0$$

Por lo tanto,

$$e^0 = 1$$

La perplejidad vale

$$1$$

Interpretación:

el punto únicamente tiene **un vecino efectivo**.

Caso 2

$$P = [0.5, 0.5]$$

La entropía es

$$H = \log 2$$

Entonces,

$$e^{\log 2} = 2$$

La perplejidad es

$$2$$

Interpretación:

el punto posee aproximadamente **dos vecinos igualmente importantes**.

Caso 3

$$P = [0.25, 0.25, 0.25, 0.25]$$

La entropía es

$$H = \log 4$$

La perplejidad resulta

$$4$$

Interpretación:

los cuatro vecinos participan con la misma importancia

La Perplejidad como objetivo del algoritmo

El usuario elige un valor de perplejidad antes de ejecutar t-SNE.

Por ejemplo,

$$\text{Perplexity} = 30$$

Este valor significa que,

en promedio,

cada observación debería considerar aproximadamente treinta vecinos efectivos.

Sin embargo,

cada observación posee una distribución de distancias diferente.

Por esta razón,

no todas pueden utilizar el mismo valor de

$$\beta.$$

Objetivo

Para cada observación,

debemos encontrar un valor de

$$\beta_i$$

tal que

$$H(P_i) = \log(\text{Perplexity})$$

Esta es exactamente la condición que resolverá la búsqueda binaria.

Correspondencia con la implementación MXNet

Observemos nuevamente el código.

```
my $desired_entropy =  
    log($desired_perplexity);
```

¿Por qué el algoritmo calcula

`log(...)`

en lugar de trabajar directamente con la perplejidad?

Recordemos la definición.

$$\text{Perplexity} = e^H$$

Aplicando logaritmos en ambos lados,

$$H = \log(\text{Perplexity})$$

Por lo tanto,

el algoritmo realmente busca satisfacer

$$H(P_i) = \log(\text{Perplexity})$$

La siguiente instrucción calcula precisamente el error cometido.

```
my $entropy_diff =  
    $entropy  
    - $desired_entropy;
```

Matemáticamente,

$$\Delta H = H(P_i) - \log(\text{Perplexity})$$

Interpretación:

- Si

$$\Delta H > 0,$$

la entropía es demasiado grande.

- Si

$$\Delta H < 0,$$

la entropía es demasiado pequeña.

En las siguientes diapositivas veremos cómo la búsqueda binaria modifica automáticamente

$$\beta$$

hasta conseguir que

$$\Delta H \approx 0.$$

¿Por qué utilizar una búsqueda binaria?

En la sección anterior vimos que el objetivo del algoritmo consiste en encontrar un valor de

$$\beta_i$$

tal que

$$H(P_i) = \log(\text{Perplexity})$$

Sin embargo,

la relación entre

$$\beta$$

y la entropía

$$H(P)$$

no puede despejarse algebraicamente.

No existe una fórmula que permita calcular directamente

$$\beta.$$

Idea

En lugar de resolver la ecuación de forma exacta,

podemos aproximar la solución mediante un procedimiento iterativo.

La técnica elegida por t-SNE es la **búsqueda binaria**.

En cada iteración,

el algoritmo ajusta el valor de

$$\beta$$

hasta que la diferencia

$$H(P) - \log(\text{Perplexity})$$

sea suficientemente pequeña.

Ejercicio 1

Cálculo manual de probabilidades, entropía y perplejidad

Considere que las distancias al cuadrado desde una observación hacia sus tres vecinos son

Vecino	Distancia²
A	2
B	8
C	18

Suponga

$$\beta = 0.25$$

Calcule paso a paso:

1. Los exponentes

$$-\beta d_i^2$$

2. Los valores

$$e^{-\beta d_i^2}$$

3. La suma

$$Z = \sum_i e^{-\beta d_i^2}$$

4. Las probabilidades normalizadas

$$P_i = \frac{e^{-\beta d_i^2}}{Z}$$

5. La entropía utilizando

$$H(P) = \log(Z) + \beta \sum_i P_i d_i^2$$

6. La perplejidad

$$\text{Perplexity} = e^{H(P)}$$

Solución del Ejercicio 1

Paso 1

Calculamos los exponentes.

Distancia²	Exponente
2	

Distancia ²	Exponente
-0.25 $\times 2 =$ -0.5	
-0.25 $8 \times 8 =$ -2	
-0.25 $18 \times 18 =$ -4.5	

Paso 2

Calculamos la exponencial.

Distancia ²	$e^{-\beta d^2}$
2	0.6065
8	0.1353
18	0.0111

Paso 3

Calculamos

$$Z = 0.6065 + 0.1353 + 0.0111 = 0.7529$$

Paso 4

Normalizamos.

Vecino	Probabilidad
A	0.8056
B	0.1797
C	0.0147

Comprobación

$$0.8056 + 0.1797 + 0.0147 = 1.0000$$

Paso 5

Calculamos

$$\sum_i P_i d_i^2$$

Vecino	$P_i d_i^2$
A	1.6112
B	1.4376
C	0.2646

Por tanto,

$$\sum_i P_i d_i^2 = 3.3134$$

Ahora,

$$H(P) = \log(0.7529) + 0.25(3.3134)$$

$$H(P) = -0.2838 + 0.8284 = 0.5446$$

Paso 6

La perplejidad es

$$e^{0.5446} = 1.724$$

Resultado final

Magnitud	Valor
Entropía	0.5446
Perplejidad	1.724

In []:

Curiosidad algebraica

¿De dónde proviene la fórmula de la entropía utilizada en MXNet?

La definición clásica de la entropía de Shannon es

$$H(P) = - \sum_i P_i \log(P_i)$$

En t-SNE,

$$P_i = \frac{e^{-\beta d_i^2}}{\sum_j e^{-\beta d_j^2}}$$

Para simplificar la escritura, definimos

$$Z = \sum_j e^{-\beta d_j^2}$$

Entonces,

$$P_i = \frac{e^{-\beta d_i^2}}{Z}$$

Paso 1

Sustituimos esta expresión en la definición de Shannon.

$$H(P) = - \sum_i \left(\frac{e^{-\beta d_i^2}}{Z} \right) \log \left(\frac{e^{-\beta d_i^2}}{Z} \right)$$

Paso 2

Aplicamos la propiedad del logaritmo

$$\log \left(\frac{a}{b} \right) = \log(a) - \log(b)$$

obteniendo

$$H(P) = - \sum_i P_i \left(\log(e^{-\beta d_i^2}) - \log(Z) \right)$$

Paso 3

Recordemos que

$$\log(e^x) = x$$

Por lo tanto,

$$\log(e^{-\beta d_i^2}) = -\beta d_i^2$$

La ecuación queda

$$H(P) = - \sum_i P_i (-\beta d_i^2 - \log(Z))$$

Paso 4

Distribuimos el signo negativo.

$$H(P) = \sum_i P_i (\beta d_i^2 + \log(Z))$$

Paso 5

Aplicamos la propiedad distributiva.

$$H(P) = \beta \sum_i P_i d_i^2 + \log(Z) \sum_i P_i$$

Paso 6

Como toda distribución de probabilidad cumple

$$\sum_i P_i = 1$$

obtenemos finalmente

$$H(P) = \log(Z) + \beta \sum_i P_i d_i^2$$

Recordando la definición de

$$Z = \sum_i e^{-\beta d_i^2},$$

la expresión final puede escribirse como

$$H(P) = \log\left(\sum_i e^{-\beta d_i^2}\right) + \beta \sum_i P_i d_i^2$$

Correspondencia con la implementación MXNet

Ahora comparemos la ecuación obtenida con el código.

Expresión matemática	Implementación MXNet
$Z = \sum_i e^{-\beta d_i^2}$	<code>my \$sum_Pi = nd->sum(\$row_P, axis => 1, keepdims => 1);</code>
$\log(Z)$	<code>nd->log(\$sum_Pi)</code>
P_i	<code>\$row_P_normalized</code>
$\sum_i P_i d_i^2$	<code>nd->sum(\$sqdistances_f64 * \$row_P_normalized, axis => 1, keepdims => 1)</code>
$\beta \sum_i P_i d_i^2$	<code>\$beta * \$sum_disti_Pi</code>

Finalmente,

```
my $entropy =
    nd->log($sum_Pi)
    + $beta * $sum_disti_Pi;
```

implementa exactamente la expresión matemática

$$H(P) = \log\left(\sum_i e^{-\beta d_i^2}\right) + \beta \sum_i P_i d_i^2.$$

Observación

Esta forma de calcular la entropía evita evaluar directamente

$$-\sum_i P_i \log(P_i),$$

lo que resulta más eficiente desde el punto de vista computacional y se adapta de forma natural a las operaciones tensoriales utilizadas por MXNet.